

Software Architecture and Design Specification

Project: File Compression Tool (Basic ZIP Implementation)

Version: 1.0

Authors: Team14

Date: 06-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	06-10-2026	Team 14	Initial SAD based on SRS Version 1.1

Approvals

Role	Name	Signature/Date
Team Member	Pranavi Sistla	
Team Member	Inchara K Kuppal	
Team Member	Pavan Kumar H K	
Team Member	Rachit Upadhyay	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the File Compression Tool (Basic ZIP Implementation). It describes the architectural structure, major components, interactions, design decisions, interfaces, and implementation considerations of the standalone C/C++ application.

1.2 Scope

The File Compression Tool is a standalone utility that uses Huffman coding for lossless file compression and decompression. The system accepts a file as input, calculates character

frequencies, constructs a Huffman tree, generates encoded bitstreams, produces a compressed file, and decompresses a previously compressed file back to its exact original state.

1.3 Audience

This document is intended for project evaluators, software developers, testers, and project team members involved in the development, testing, evaluation, and maintenance of the system.

1.4 Definitions

CLI: Command Line Interface used to interact with the tool.

Huffman Coding: A lossless data compression algorithm that assigns variable-length codes to input characters.

Huffman Tree: The tree structure used to generate Huffman codes based on character frequencies.

Compression: The process of reducing the size of an input file using Huffman coding.

Decompression: The process of restoring a compressed file to its original contents.

ZIP: The project name refers to the basic file compression tool. The implementation uses Huffman-based compression and a custom compressed-file format; it does not claim compatibility with standard ZIP archives.

2. Document Overview

2.1 How to use this document

This document provides the architectural and design details of the File Compression Tool (Basic ZIP Implementation). It should be used by developers as a reference for implementing the system components and their interactions. Testers and evaluators can use it to understand the system structure, interfaces, and design decisions while verifying the implemented requirements

2.2 Related Documents

Software Requirements Specification (SRS) – File Compression Tool (Basic ZIP Implementation), Version 1.1

Software Test Plan (STP) – File Compression Tool (Basic ZIP Implementation)

Requirements Traceability Matrix (RTM)

3. Architecture

3.1 Goals & Constraints

Goals:

- Provide lossless file compression using Huffman coding.
- Support decompression of previously compressed files to their exact original contents.
- Correctly calculate character frequencies and construct the Huffman tree.
- Generate encoded bitstreams and produce compressed output files.
- Provide reliable file input and output through the command-line interface.
- Maintain byte-for-byte correctness during decompression.
- Support files up to 100 MB within the specified performance requirement.
- **Performance:** The system shall support compression and decompression of text and binary files up to 100 MB within a target processing time of under 10 seconds on a standard modern processor.
- **Memory Efficiency:** The system shall maintain memory consumption within the size of the file being processed plus the overhead required for the Huffman tree.

Constraints:

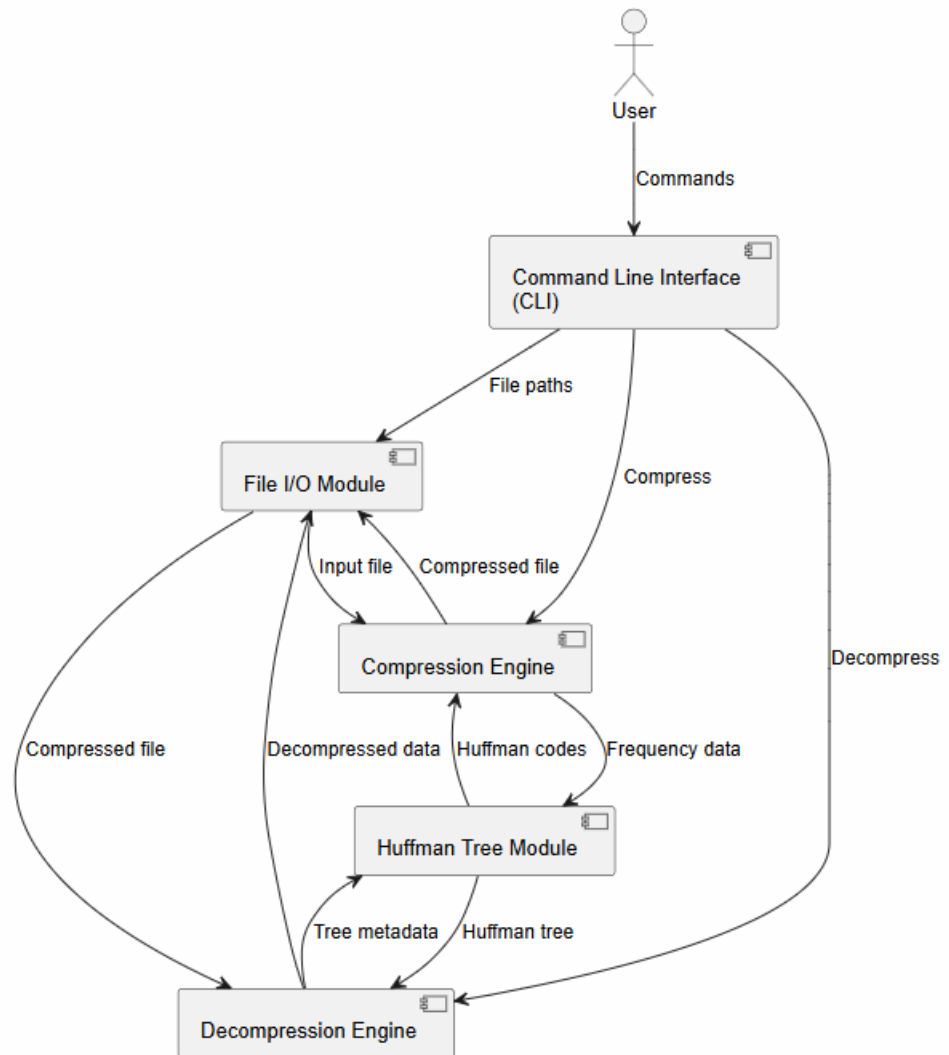
- The system must be developed strictly using C/C++.
- Huffman coding must be used as the compression algorithm.
- The application is a standalone utility operated through a Command Line Interface (CLI).
- Standard C/C++ libraries are used for file input and output.
- The system depends on the host operating system's file permissions and available disk space.
- The system should support Windows, Linux, and macOS environments with C/C++ compilation and execution.

3.2 Stakeholders & Concerns

Stakeholder	Main Concerns
General Users	Easy file compression and decompression
Developers	Correct Huffman implementation and maintainability
Testers	Functional correctness and lossless output
Project Evaluators	Requirement compliance and performance
System Administrators/Developers	Reliable and efficient file processing

3.3 Component (UML) Diagram

File Compression Tool - Component Diagram



3.4 Component Descriptions

CLI: Accepts compression/decompression commands and file paths.

File I/O Module: Reads input files and writes compressed/decompressed files.

Compression Engine: Calculates character frequencies, constructs the Huffman tree, generates encoded bitstreams, and calculates compression metrics including original file size, compressed file size, and compression ratio.

Huffman Tree Module: Builds and provides the Huffman tree and associated metadata.

Decompression Engine: Extracts metadata and decodes the compressed bitstream.

3.5 Chosen Architecture Pattern and Rationale

A **Layered Architecture** is selected to separate user interaction, file handling, compression/decompression processing, and Huffman logic. This provides clear separation of responsibilities and makes the system easier to develop, test, and maintain.

3.6 Technology Stack & Data Stores

- Language:** C/C++
- Interface:** Command Line Interface
- File Handling:** Standard C/C++ I/O libraries
- Compression Algorithm:** Huffman Coding
- Data Storage:** Local file system
- Database:** Not required
- External Libraries:** Not required

Performance Constraint: The implementation shall support files up to 100 MB and target compression and decompression within 10 seconds. Memory usage shall remain within the file size plus Huffman tree overhead.

3.7 Risks & Mitigations

Risk		Mitigation	
Corrupted compressed file		Validate file data and terminate gracefully	
Insufficient disk space		Report error without corrupting existing data	
Incorrect Huffman implementation		Test tree construction and encoded data	
Loss of original data		Verify byte-for-byte decompression	
Large file processing time		Test performance up to 100 MB	

3.8 Traceability to Requirements

Requirement		Architecture Component	Design Reference	Code File Reference
REQ-1		CLI / File I/O Module	3.3, 3.4, 4.3	TBD

Requirement	Architecture Component	Design Reference	Code File Reference
REQ-2	Compression Engine	3.3, 3.4, 4.1	TBD
REQ-3	Huffman Tree Module / Compression Engine	3.3, 3.4, 4.1, 4.2	TBD
REQ-4	File I/O Module	3.3, 3.4, 4.3	TBD
REQ-5	Decompression Engine	3.3, 3.4, 4.1, 4.2	TBD
REQ-6	Decompression Engine / Huffman Tree Module	3.3, 3.4, 4.1, 4.2	TBD
REQ-7	File I/O Module	3.3, 3.4, 4.1, 4.2	TBD

3.9 Security Architecture

Security Objectives

- **File Protection:** Ensure that file operations respect the operating system's read/write permissions and do not bypass system-level security controls.
- **Safe File Processing:** Ensure that corrupted or invalid compressed files are handled safely without causing crashes or corrupting existing user data.

Security Requirements

- The system shall operate within the file permissions provided by the host operating system and shall not bypass system-level read/write security protocols.
- The system shall safely handle corrupted files and file-operation failures and shall terminate gracefully without corrupting existing user data.

Security Measures

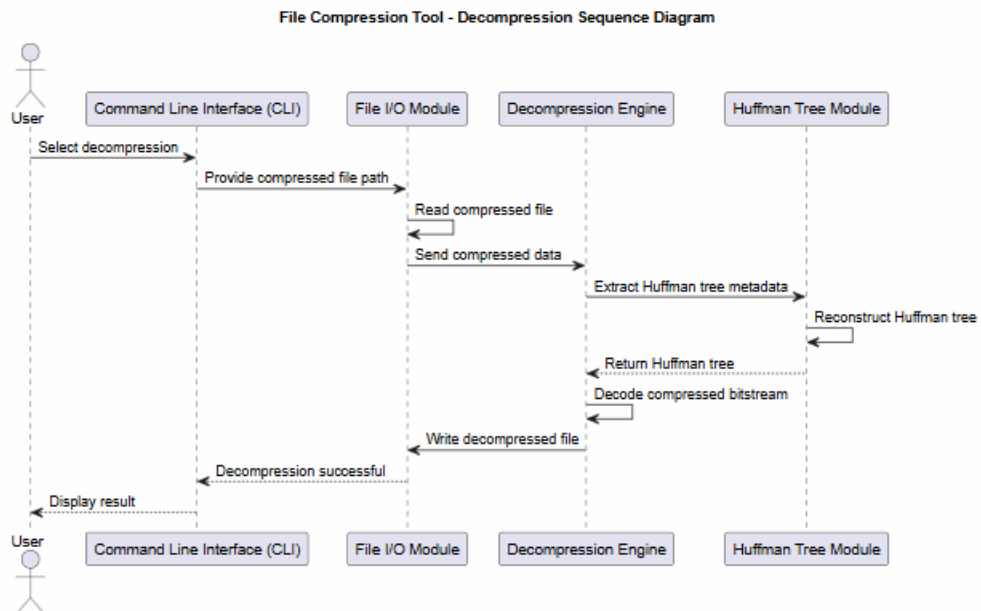
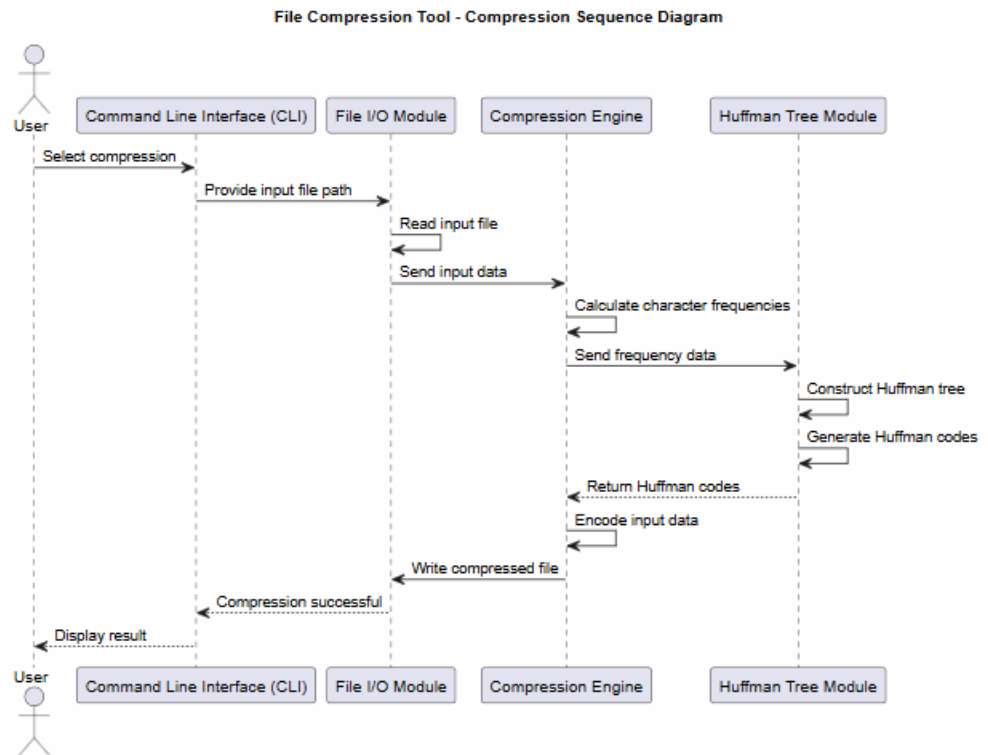
- Input and output file operations shall use standard operating-system permissions.
- Compressed files shall be validated before their contents are processed during decompression.
- Corrupted or incomplete compressed data shall be detected and handled gracefully.
- File-writing operations shall avoid overwriting or corrupting existing user data when an operation fails.

4. Design

4.1 Design Overview

The system is designed as a standalone C/C++ command-line application that implements Huffman-based lossless file compression and decompression. The compressed output uses a custom format containing the information required to reconstruct the Huffman tree and decode the compressed bitstream. The implementation does not claim compatibility with standard ZIP archive specifications.

4.2 UML Sequence Diagrams



4.3 API Design

The system does not use REST or web APIs. Its primary interface is the **Command Line Interface (CLI)**.

Interface	Function
CLI	Accept compression/decompression commands and file paths
File I/O	Read input files and write output files
Compression Engine	Process frequency data and encode files
Decompression Engine	Decode compressed files and restore data
Compression Metrics	Compression Engine / CLI

Compression Metrics: After a successful compression operation, the system shall provide the original file size, compressed file size, and compression ratio through the CLI.

4.4 Error Handling, Logging & Monitoring

The system shall handle invalid file paths, file access failures, corrupted files, and insufficient disk space gracefully. Appropriate error messages shall be displayed through the Command Line Interface (CLI). The system shall terminate safely without crashing or corrupting existing user data.

The system shall monitor processing time and file size measurements to evaluate performance during compression and decompression.

4.5 UX Design

The system uses a simple **Command Line Interface**. Users provide file paths and compression or decompression commands through the console. Standard error messages are displayed for invalid inputs and file access errors.

4.6 Open Issues & Next Steps

Implement the Huffman tree and encoding logic.

Implement compressed-file metadata handling.

Implement decompression and lossless restoration.

Validate file handling and error conditions.

Test performance for files up to 100 MB.

Verify byte-for-byte correctness of decompressed files.

Edge Case Handling: Empty input files shall be handled without unexpected termination. Files containing only one repeated character shall be processed using an appropriate Huffman representation and shall be decompressed to the original contents. Corrupted or incomplete compressed files shall be detected during decompression and handled gracefully without corrupting existing user data.

5. Appendices

5.1 Glossary:

CLI: Command Line Interface used to interact with the tool.

Huffman Coding: A lossless data compression algorithm using variable-length codes.

Huffman Tree: Tree structure used to generate Huffman codes.

Compression: Process of reducing file size using Huffman coding.

Decompression: Process of restoring compressed data to its original form.

ZIP: The project name for the basic file compression tool. The implementation uses a custom Huffman-based compressed-file format and does not claim compatibility with standard ZIP archives.

SRS: Software Requirements Specification.

SAD: Software Architecture and Design Specification.

REQ: Requirement identifier used in the SRS.

5.2 References:

- Software Requirements Specification (SRS), File Compression Tool (Basic ZIP Implementation), Version 1.1.
- Huffman, D. A. (1952), *A Method for the Construction of Minimum-Redundancy Codes*.
- PKWARE Inc., .ZIP File Format Specification.
- ISO/IEC 14882:2020, C++20 Standard. SRS_team_14_SE_mini_project

5.3 Tools

C/C++ Compiler

Standard C/C++ Libraries

PlantUML / draw.io for UML diagrams

Microsoft Word for documentation

Git/GitHub for version control